

AutoBook

Zadanie 2 · Checkpoint Dokumentácia

Andrii Dosyn Anton Hrimov

Obsah

1	Špecifikácia zadania a Zámer	2
2	Deklarácia použitých OOP princípov	2
2.1	Generalizácia (Generalization)	2
2.2	Agregácia (Aggregation)	3
2.3	Kompozícia (Composition)	3
2.4	Dedenie: Dve hierarchie (Inheritance)	3
2.5	Polymorfizmus (Compile-time a Run-time)	5
2.6	Preťaženie (Overloading)	5
2.7	Prekonanie (Overriding)	5
3	Diagram tried entít	6
4	AI Mikroslužba	6
4.1	Architektúra a storage mikroservisu	7
4.2	Zapuzdrenie (Encapsulation)	7
4.3	Princíp jednej zodpovednosti (SRP)	8
5	Unit testy a Integrácia	8
6	Implementácia používateľského rozhrania	8
6.1	Hlavná stránka (Feed)	9
6.2	Knižnica (Library) a profil	10
6.3	Detail knihy a Príspevku	11
6.4	Pokročilý editor a edit requests	12
7	Používateľský manuál	13
8	Záver	13

1 Špecifikácia zadania a Zámer

AutoBook je platforma fungujúca pre autorov a čitateľov. Spája komunitu ľudí a poskytuje inteligentný analytický nástroj pre autorov – **Vocabulary Builder** – s podporou umelej inteligencie (NLP). Projekt bol rozvrstvený na hlavnú Java Spring Boot aplikáciu a Python FastAPI mikroslužbu zodpovednú za analytiku.

Hlavné funkcionality pre Checkpoint (základná implementácia):

- Používateľský systém, publikovanie kníh a písanie príspevkov v rámci kompozitnej hierarchie z backend prístupu.
- Interakcia formou komentárov a sledovania (*Follow*) autorov.
- Generovanie slovníka (Vocabulary).
- Služby implementované primárne v Jave nad databázou, prepájajúce entitné formáty pomocou JPA.

2 Deklarácia použitých OOP princípov

Zdrojový kód systému bol navrhnutý podľa princípov objektovo-orientovaného programovania, s dôrazom na oddelenie logiky a vrstiev dát.

2.1 Generalizácia (Generalization)

Uplatnili sme princíp generalizácie tak pri konštrukcii rozhraní, ako aj v Java výnimkách. Dobrým príkladom je dedenie vlastných výnimiek od všeobecného typu `RuntimeException`, alebo využívanie všeobecného abstraktného typu v AI prístupe k úložisku. Kód ukazuje generalizovaný pohľad na odchyťovanie chýb priamo v business vrstve aplikácie (`Exception` balík v Jave).

```
package com.autobook.Exception;

public class BookNotFoundException extends RuntimeException {

    public BookNotFoundException(Long bookId) {
        super("Book not found with id: " + bookId);
    }
}
```

2.2 Agregácia (Aggregation)

V Java entitách sme nastavili agregáciu prostredníctvom voľných kompozitných väzieb. Napríklad rola **Autora** a **Knihy** v balíku ‘com.autobook.Library.Book.Book’. Kniha uchováva objekt ‘User’ (autora) pomocou anotácie rozhrania ‘@ManyToOne’. Používateľ bez nej dokáže nezávisle naďalej existovať a uplatňovať svoju entitnú logiku inde na sieti a v príspevkoch.

```
@Entity
@Table(name = "books")
@Getter
@Setter
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User author;
```

2.3 Kompozícia (Composition)

Silnú väzbu s plným riadením životného cyklu demonštruje vzťah **Post** a **Comment**. Komentáre sú existenčne viazané na príspevok pod ktorým visia. V entitnom kóde `Post.java` je použitá kaskáda typu `CascadeType.ALL` a parameter `orphanRemoval = true`, čo fyzicky vynucuje OOP kompozíciu brániacu úniku dát z pamäte.

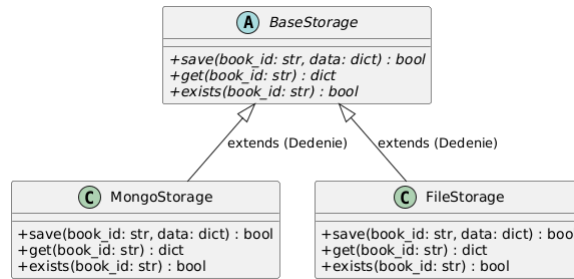
```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Comment> comments = new ArrayList<>();
```

2.4 Dedenie: Dve hierarchie (Inheritance)

Projekt striktne uplatňuje systém kódovania do hierarchií, aby sme zachovali rozšíriteľnosť pre budúcnosť. Uplatňujeme nasledovné dve preukázateľné dedičné cesty:

1. Hierarchia v dátovom formáte (*Python mikroslužba*)

Úložisko tvorené rodičom ‘BaseStorage’ určujúcim zmluvný kontrakt dovoľuje poddediť triedy implementátorov pre databázu ‘MongoStorage’ a lokálne úložisko pre disky ‘FileStorage’.



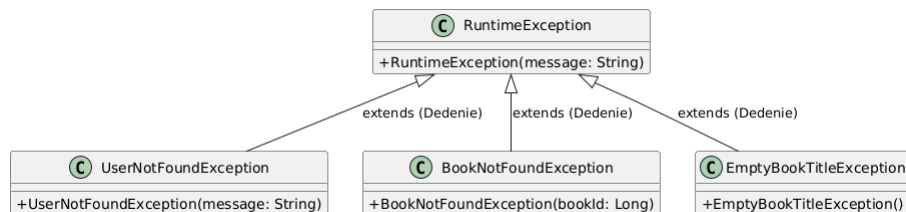
Obr. 1: Dedenie prvej hierarchie: Polymorfne abstraktné úložisko.

```

class MongoStorage(BaseStorage):
    def __init__(self):
        self._client = None
        self._collection = None
        self._connect()
    
```

2. Hierarchia chýb (*Java Autorské Výnimky*)

Druhú dedičnú vetvu reprezentuje balík s výnimkami. Každé porušenie biznis logiky (neexistencia knihy, používateľa, zlé id) prediedla priamo z nadradenej java triedy `RuntimeException`.



Obr. 2: Dedenie druhej hierarchie: Hierarchia Java výnimiek `RuntimeException`.

```

package com.autobook.Exception;

public class BookNotFoundException extends RuntimeException {

    public BookNotFoundException(Long bookId) {
        super("Book not found with id: " + bookId);
    }
}
    
```

2.5 Polymorfizmus (Compile-time a Run-time)

Polymorfizmus za behu (Run-time overriding):

Uplatnený najviac pri prístupoch nad úložiskom v Python mikroservise, kde samotná analytická logika poberá v úvode polymorfný dcérový objekt (napr. `MongoStorage`), no posíela mu dopyt nad fasádou abstraktnej volajúcej metódy rodiča z inštancie `'BaseStorage'`.

Polymorfizmus pri preklade (Compile-time overloading):

Typicky prítomný na repozitárnych dopytoch s dynamickým *overloadingom* pri preťažení. Napríklad dopyty `findByAuthor(User author)` vs. `findByAuthorAndPrivacy(User author, PrivacyType privacy)` v `BookService.java` simulujú zmenený signatúrny model vyhľadávania nad knihami v jedinom kontexte preťaženia rovnakého požiadavku.

```
public List<Book> getBooksByAuthor(User author) {  
    return bookRepository.findByAuthor(author);  
}  
  
public List<Book> getBooksByPrivacy(PrivacyType privacy) {  
    return bookRepository.findByPrivacy(privacy);  
}
```

2.6 Preťaženie (Overloading)

Preťažovanie doménových konštruktorov v javovskej knižnici výnimok. Napríklad vyvolanie inštancie z `RuntimeException(String)` namiesto prázdnej konštrukcie obohacuje stav objektu metódami poslenými počas kompilácie. Toto generuje konštrukcie obalov na chybové javy.

```
public Book getBookById(Long bookId) {  
    return bookRepository.findById(bookId)  
        .orElseThrow(() -> new BookNotFoundException(bookId));  
}
```

2.7 Prekonanie (Overriding)

Použitie a priame dedičné prekonávanie metód realizujeme v abstraktnej databáze `'BaseStorage.save()'` (v Pythone cez `'@abstractmethod'`), ktorej implementačný prístup `MongoStorage` povinne realizuje (*Overriding* správania). V Jave samotné výnimky prepisujú svoj stavový výpis chýb posúvaním chybovej inštancie priamo do rodičovskej triedy (výpis `'super(msg)'`).

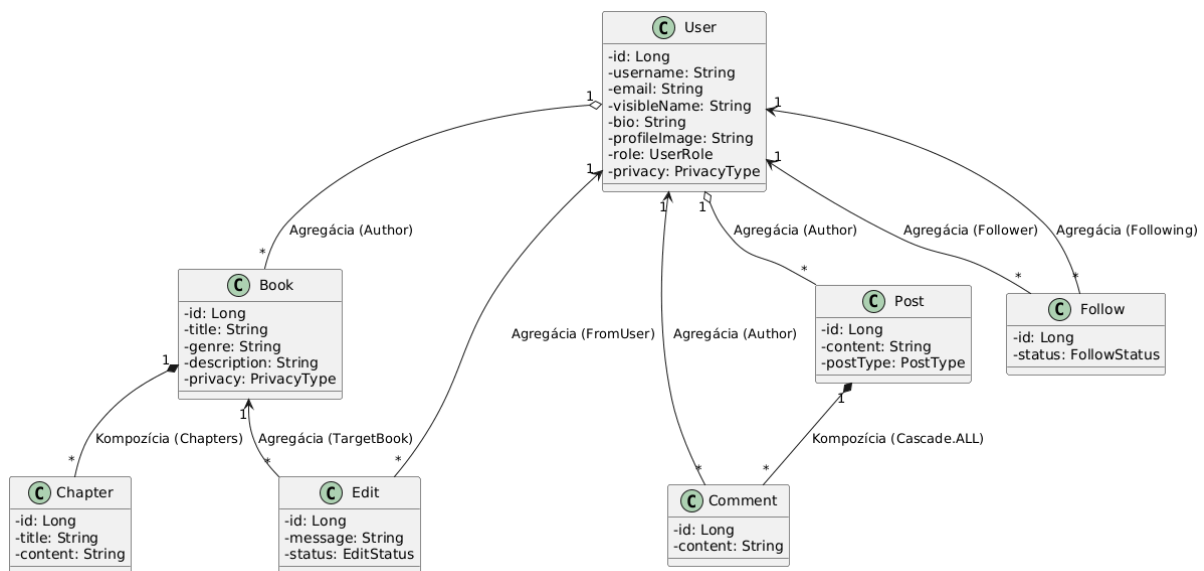
```
package com.autobook.Exception;

public class BookNotFoundException extends RuntimeException {

    public BookNotFoundException(Long bookId) {
        super("Book not found with id: " + bookId);
    }
}
```

3 Diagram tried entít

Dátová a logická previazanosť základnej doménovej vrstvy aplikácie v Jave zahŕňa modely užívateľov, publikácií, komentárov, označení a žiadostí na editáciu.



Obr. 3: Kompletný diagram zdôrazňujúci asociácie, agregáciu a krížovú kompozíciu repozitárnych Java entít.

4 AI Mikroslužba

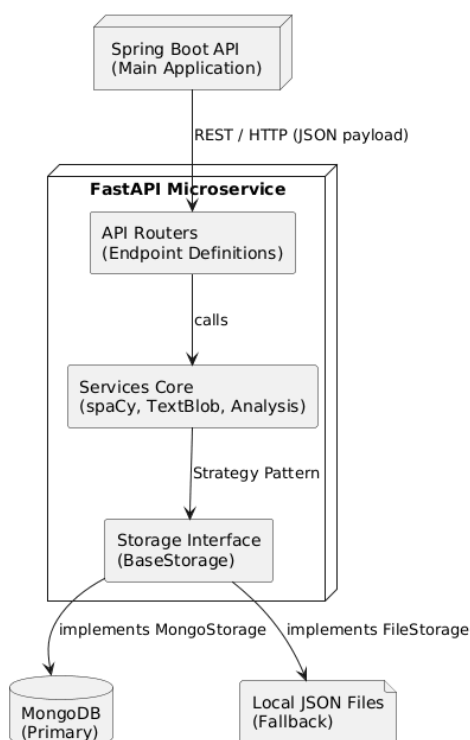
AutoBook AI je samostatný mikroservis zameraný na analýzu textu pomocou metód spracovania prirodzeného jazyka (NLP). Služí ako inteligentné **analytické jadro** platformy. Z decentralizácie vyplýva uvoľnenie záťaže hlavnej Java (Spring Boot) aplikácie.

4.1 Architektúra a storage mikroservisu

Aplikácia je postavená ako REST API pomocou asynchrónneho frameworku **FastAPI** v jazyku **Python**. Python bol zvolený pre výborný komunitný prístup k NLP knižniciam:

- **spaCy** – Zabezpečuje lemmatizáciu a hľadanie štrukturálnych autorských fráz v angličtine.
- **TextBlob** – Vyhodnocuje "sentiment" (emócie a subjektivitu vnútornej stavby kapitol).

O ukladanie stavov sa stará dokumentová vrstva nad **MongoDB**, čím dokážeme zachovať polia metrík s variabilnou JSON dĺžkou. Ako úložisko pre vývoj slúži *File Storage*.



Obr. 4: Decentralizovaná architektúra mikroslužby zaťaženej AI operáciami.

4.2 Zapuzdrenie (Encapsulation)

Analytické triedy (napr. `StyleConsistencyAnalyzer`) vo vysokej miere využívajú zapuzdrenie. Komplexná matematika je pre vonkajší kód zakrytá v bezpečných chránených metódach (ako `_lexical_similarity` a `_phrase_coverage`).

4.3 Princíp jednej zodpovednosti (SRP)

Architektúra rozdeľuje logiku tak, aby kontrolery pre API dopyty nikdy priamo nerátali analytiku, a naopak analytické enginy nepoznajú vrstvu HTTP zberu dát formou *Routers* → *Services* → *Models*.

5 Unit testy a Integrácia

V súčasnosti je aplikácia dôkladne pokrytá testami pre dôležité služby. Logika v **Java** je prítomná v testovacom súbore `BookServiceTest.java` využívaním **JUnit 5** a knižnice **Mockito**. Pokryté je vstrekovanie simulovaných závislostí repozitárnych implementácií ako ‘InjectMocks’ a ‘Mock’.

Popis kľúčových Unit testov (Java):

1. `createBook_ok` a `createBook_emptyTitle`: Overujú správanie vytvorenia knižnej publikácie po prijatí správneho obsahu oproti zamietnutiu (vyvolaniu explicitnej výnimky `EmptyBookTitleException`) v prípade prázdneho názvu.
2. `getBookById_ok` a `getBookById_notFound`: Testujú Optional prístup z databázy. Ak UUID neexistuje, `assertThrows` potvrdí vyhodenie výnimky `BookNotFoundException`.
3. `updateBook_ok`: Overuje správanie prikázanej editácie entít so simulovanými zmenami vlastností od klienta.

Súčasný Python scripty v AI analytike sa stavajú formou integračných jednotiek skrz **pytest**. Popis kľúčových Unit testov (Python):

1. `test_routes`: Overuje endpointy FastAPI, správnosť prenosu JSON štruktúr nad aktívnym routerom a testuje HTTP stavové kódy odozvy mikroslužby pri zaslaní analytických textových dopytov.

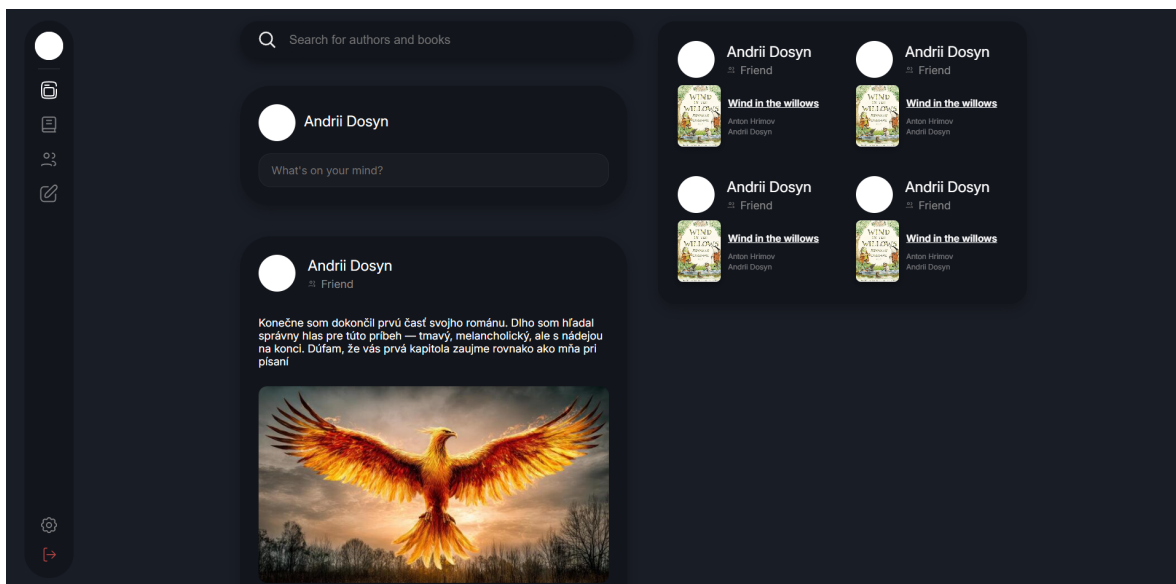
6 Implementácia používateľského rozhrania

Nasledujúce obrazovky predstavujú reálnu funkčnú implementáciu grafického rozhrania (frontend) systému AutoBook. Podrobný manuál ako systém *spustiť* sa nachádza v priloženom `README.md` v koreni repozitára.

Použité technológie (Frontend stack)

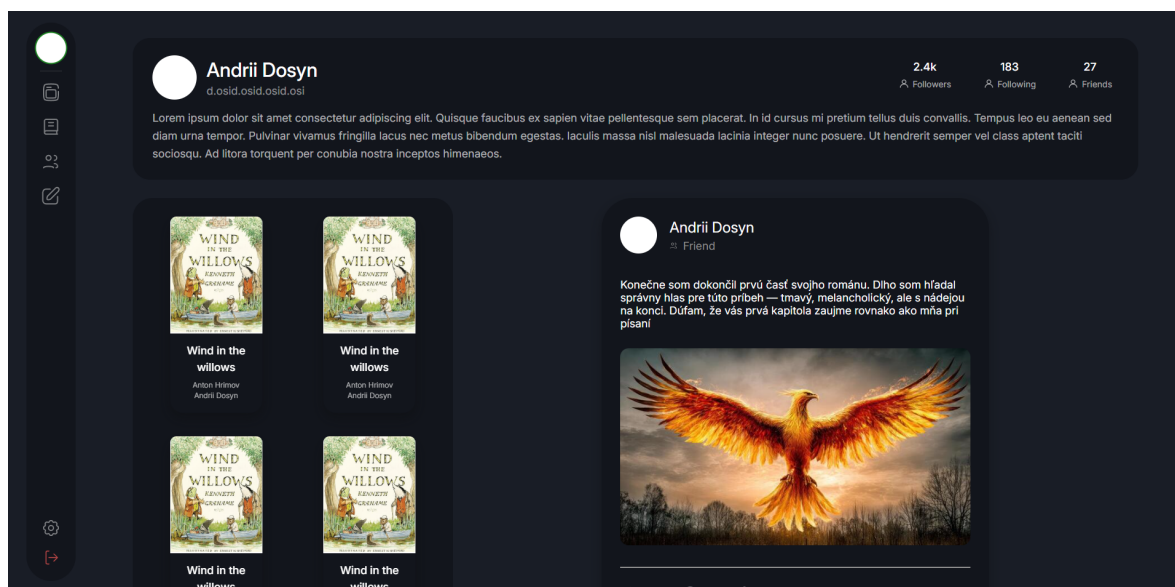
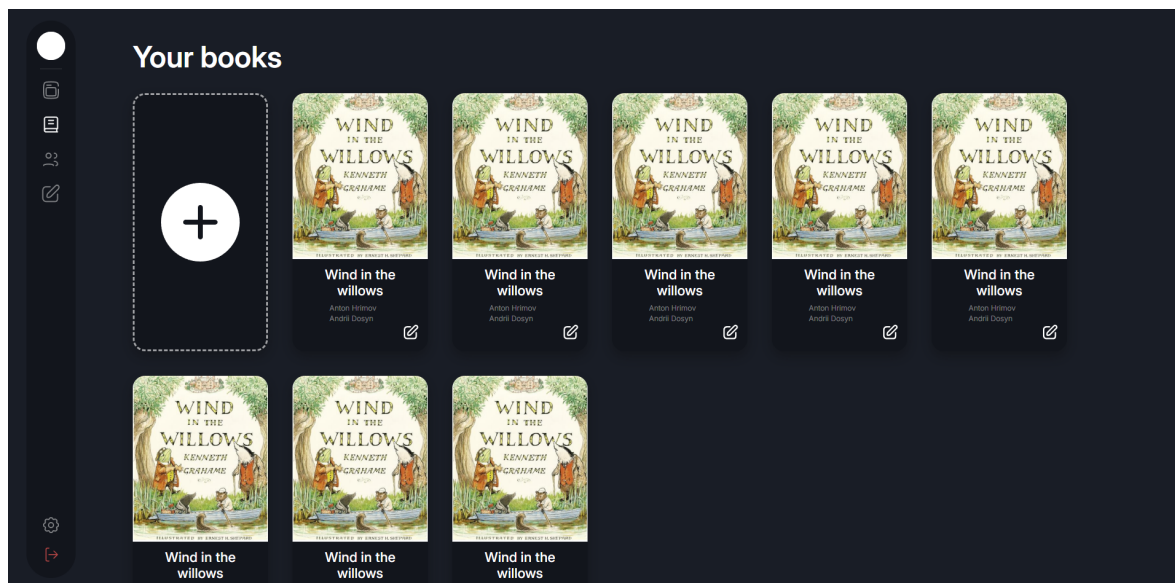
Rozhranie je plne vyvinuté v **TypeScripte** pomocou **React**. Pre štylizáciu bol zvolený plynulý Dark Mode. Aplikácia využíva klientske smerovanie (React Router) a robustný textový editor **Tiptap**.

6.1 Hlavná stránka (Feed)



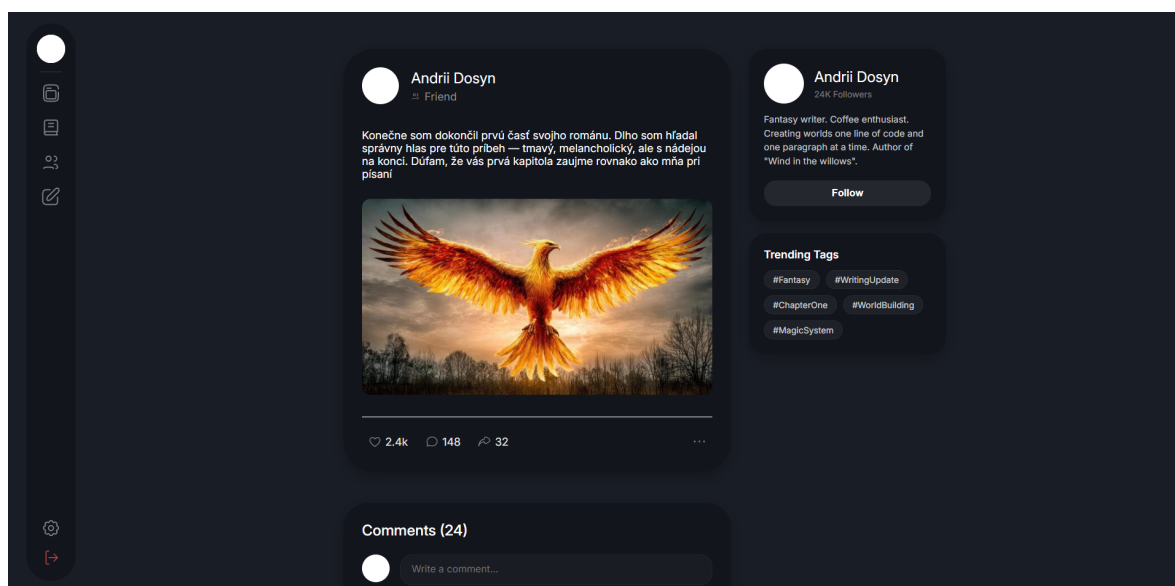
Hlavná informačná os zobrazujúca aktivitu siete. Ponúka inline formuláre pre sociálne interakcie.

6.2 Knižnica (Library) a profil



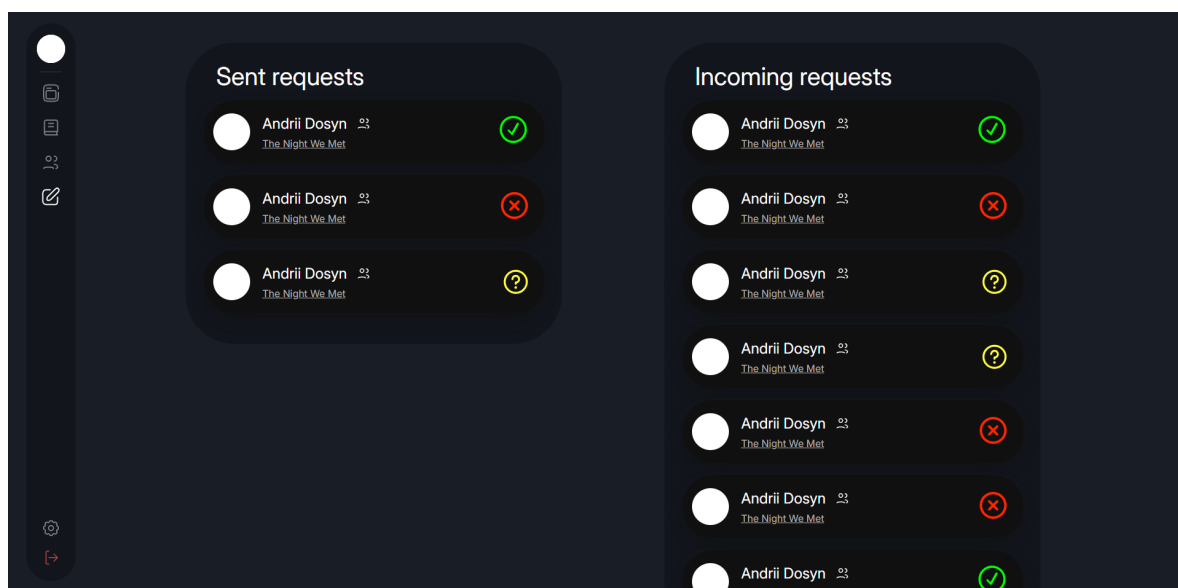
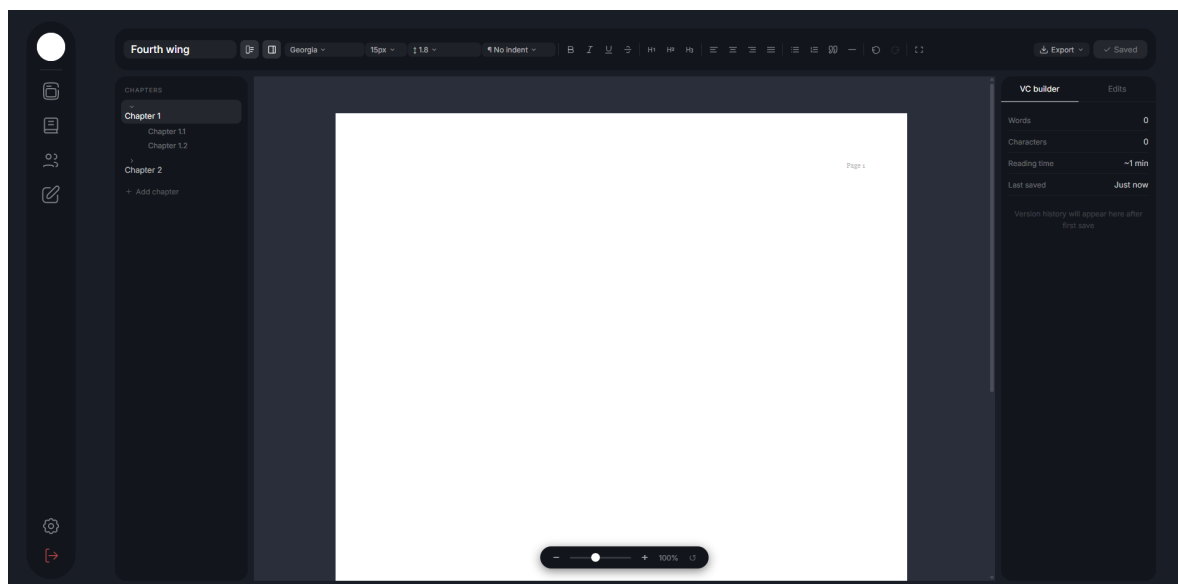
Profil zhromažďuje autorove informácie, zatiaľ čo Library slúži na správu rozpracovaných kníh.

6.3 Detail knihy a Príspevku



Stránky využívajú prepracovaný dvojtĺpcový dizajn a implementujú pokročilú stromovú štruktúru komentárov.

6.4 Pokročilý editor a edit requests



Současťou platformy je profesionálny editor stránkovaný na vizuálny formát A4 (Focus mode). Autori tu spravujú návrhy na úpravu od čitateľov.

7 Používateľský manuál

Z dôvodu optimalizácie veľkosti repozitára nie sú na GitHubu zaradené inštalované knižnice a systémové moduly (`node_modules`, Python virtual env). Služby je potrebné pred prvým spustením zinicializovať oddelene:

1. Frontend používateľské rozhranie (React)

Pre úspešné spustenie je nutné mať v systéme nainštalované prostredie **Node.js** (aspoň verziu 18). Architektúra klienta stavia na modernom build-systéme **Vite** a aktívne preinštalováva pokročilé knižnice (ako `react-router-dom` na smerovanie, `axios` pre API, textový balík `tiptap` a nástroje na PDF export ako `jspdf`). Nasmerujte terminál do priečinka `/frontend`. Najskôr nainštalujte závislosti s použitím Node balíčkovace a následne spustíte lokálny dev-server cez **Vite**.

- `npm install` (doinštaluje závislosti z `package.json` nad repozitárom)
- `npm run dev` (zapne platformu cez bleskový Vite server)

2. Asynchrónna AI Mikroslužba (Zadanie2_fixed)

Pracovný Python backend beží nad balíkom FastAPI. Prejdite terminálom do koreňa Python mikroslužby a zavolajte:

- `pip install -r requirements.txt` (stiahnutie knižníc vrátane `spaCy`)
- `python -m spacy download en_core_web_sm` (povinný sťahovač en modelov)
- `uvicorn main:app -reload` (finálne lokálne spustenie servera)

8 Záver

Úspešná integrácia rozsiahlej Spring Boot infraštruktúry spoločne z Python AI analytickým modulom vytvorila silný predpoklad pre funkčnú sociálnu sieť pre autorov a čitateľov. Aplikovaním overených OOP princípov sa ukázala pripravenosť architektúry na ďalšie škálovanie, pridávanie nových perzistentných vrstiev a pokročilé napájania na umelú inteligenciu. Dátový model bezpečne pracuje so vzťahmi nad databázami a vizuálna ukážka odráža progres naplnenia základnej používateľskej cesty definovanej na začiatku projektu.